

Solution Requirements

Date	14 July 2026
Team ID	SWTID-2026-7049
Project Name	Emotion Detection & Learning Support Engine (AI Learning Assistant)
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements

Functional Requirements (FR) define specific behaviors, functions, or features of the system. Non-Functional Requirements (NFR) specify how well the system performs those behaviors. Every row below reflects a requirement the built system actually satisfies, verified against code and the project's own test/analysis artifacts rather than aspirational scope.

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Emotion Detection	Classify a student's free-text description of a study problem into one of five emotion classes (Bored, Confident, Confused, Curious, Frustrated) using both a BiLSTM and a BERT model.	High
2	Trust-Adaptive Decision	Combine the two models' outputs using a rule-based decision engine that returns a final emotion, a trust level (HIGH / MEDIUM / LOW), and a machine-readable reason code.	High
3	AI-Generated Guidance	Generate a supportive, field-aware response via an LLM (Gemini or OpenRouter), tailored to the detected emotion and field of study, with a template-based fallback when AI generation is disabled or unavailable.	High
4	Session Settings	Let the user independently toggle AI response generation, CSV data saving, and detail visibility (Model Comparison / How I Decided) per session.	Medium
5	Interaction Logging	Record each interaction (field, problem, decision, AI response, model scores) to a CSV log when the user opts in, timestamped in IST.	Medium
6	Analytics Dashboard	Visualize accumulated session interactions as an emotion-distribution pie chart, an emotional-journey timeline, and a per-field emotion breakdown bar chart.	Medium
7	Explainability	Optionally display the raw BiLSTM/BERT predictions side by side and the decision engine's reasoning ("How I Decided") for the current result.	Medium
8	Graceful Degradation	Continue operating with reduced functionality (BiLSTM-only, template responses) if BERT weights or the LLM API are unavailable, rather than failing outright.	High

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	End-to-end analysis (both model inferences + AI response) should complete within a single user-perceived wait, shown via a loading spinner.	Validated in automated E2E testing: 10 real-weight, real-API test runs completed in ~224s total (~22s/interaction avg).
2	Model Accuracy	The emotion classifier should perform materially better than chance (20% for 5 balanced classes) on held-out data.	BERT: 83% accuracy / 0.83 weighted F1 on a 770-row validation set. BiLSTM: 53% accuracy / 0.58 weighted F1 (analysis/decision_signals_report.txt).

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
3	Reliability & Availability	The app must not crash if a model or the AI provider is unavailable.	Verified via DISABLE_BERT / DISABLE_AI_RESPONSE flags; the public Streamlit Cloud deployment runs live in exactly this degraded mode.
4	Process Isolation	TensorFlow and PyTorch/transformers must never be loaded in the same process (confirmed to cause a hard segfault).	Each model loads in its own isolated subprocess; verified via scripts/_integration_check_bilstm.py, _integration_check_bert.py, and envtesting/test_isolation_*.py (3/3 passed locally).
5	Usability	Settings and results should read as one linear flow rather than scattered widgets.	Input → Analyze → Decision → AI Guidance → Model Comparison → Explainability; confirmed via a 10/10-passing Streamlit AppTest suite plus a manual click-through (VALIDATION_REPORT.md).
6	Maintainability	Core decision logic should be pure/testable, independent of the UI framework.	decision_engine.py has zero Streamlit/file/network calls by design (documented in its own module docstring), so it can be unit tested and swapped out if models are retrained.